Ingénieur 2 – Apprentissage
Majeure Logiciels et Système d'information
Promotion 2027

ALTN83-2526BSP01 - Bases de données réparties (INGE-2-APP-LSI-B)

Projet base de données – Nano Orbit

Rémi le guernrnevé, Tom AUXERRE, Thomas ADRAGNA

Table des matières

1 – Phase 1 : Conception & Architecture distribuée *3*

1.1 – Étape 1 : Dictionnaire de données et classification **3**

1.1.1 – Table Satellite 3
1.1.2 – Orbite 4
1.1.3 – Instrument 5
1.1.4 – Embarquement 5
1.1.5 – Station_Sol 6
1.1.6 – Centre_Controle 6
1.1.7 – Affectation Station 7
1.1.8 – Mission 7
1.1.9 – Fenêtre_com 8
1.1.10 – Participation 8

1.2 – Étape 2 : Modèle Conceptuel de Données (MCD) **9**

1.3 – Étape 3 : Modèle Logique de Données (MLD) **9**

1.4 – Étape 4 : Réflexion sur l'architecture distribué **10**

1.4.1 – Quelles tables du MLD sont strictement locales à un centre de contrôle ? 10
1.4.2 – Quelles tables doivent être globales (accessibles depuis tous les centres) ? 10
1.4.3 – Comment le centre de Singapour peut-il continuer à planifier des fenêtres de communication si le serveur central est indisponible ? 11
1.4.4 – Quels risques de cohérence identifiez-vous dans ce système multi-sites ? 12

1 – Phase 1 : Conception & Architecture distribuée

1.1 – Étape 1 : Dictionnaire de données et classification

1.1.1 – Table Satellite

Attribut	Code	Type Oracle	Obligatoire	Unique	Contraintes / Remarques
Identifiant satellite	id_satellite	VARCHAR2(20)	OUI	OUI	Format SAT-XXX, immuable (RG-S01)
Nom	nom_satellite	VARCHAR2(100)	OUI	NON	
Date lancement	date_lancement	DATE	OUI	NON	
Masse	masse	NUMBER(5,2)	OUI	NON	> 0
Format CubeSat	format_cubesat	VARCHAR2(5)	OUI	NON	1U / 3U / 6U / 12U
Statut	statut	VARCHAR2(30)	OUI	NON	Opérationnel / En veille / Défaillant / Désorbité
Durée de vie	duree_vie_prevue	NUMBER(4)	OUI	NON	
Batterie	capacite_batterie	NUMBER(6,1)	OUI	NON	
Orbite associée	id_orbite	VARCHAR2(20)	OUI	NON	FK vers ORBITE

Code	Règle de gestion (description complète)	Catégorie	Implémentation Oracle
RG-S01	Chaque satellite est identifié de manière unique par un code alphanumérique (ex. SAT-001). Ce code est immuable après la mise en orbite et ne peut pas être modifié.	Structure relationnelle	PK + gestion applicative (immutabilité)
RG-S02	Un satellite est associé à une orbite à un instant donné. Au cours de sa vie, il peut changer d'orbite, mais seule l'orbite courante est conservée dans le système.	Structure relationnelle	FK vers ORBITE
RG-S03	Un satellite peut embarquer entre 1 et 4 instruments. Un même modèle d'instrument peut équiper plusieurs satellites différents.	Structure relationnelle	Association N-N + CHECK (cardinalité)
RG-S04	Pour chaque instrument embarqué sur un satellite, on enregistre la date d'intégration et l'état de fonctionnement. Ces attributs dépendent du couple satellite-instrument.	Structure relationnelle	Table associative EMBARQUEMENT

RG-S05	Un satellite doit participer à au moins une mission. Une mission peut mobiliser plusieurs satellites simultanément.	Structure relationnelle	Association N-N PARTICIPATION
RG-S06	Un satellite ayant le statut « Désorbité » ne peut plus être assigné à une nouvelle mission ni générer de nouvelles fenêtres de communication.	Mécanisme procédural	Trigger BEFORE INSERT

1.1.2 – Orbite

Attribut	Code	Type Oracle	Obligatoire	Unique	Contraintes / Remarques
Identifiant orbite	id_orbite	VARCHAR2(20)	OUI	OUI	
Type	type_orbite	VARCHAR2(10)	OUI	NON	LEO / SSO / etc
Altitude	altitude	NUMBER(5)	OUI	NON	
Inclinaison	inclinaison	NUMBER(5,2)	OUI	NON	
Période	periode_orbitale	NUMBER(6,2)	OUI	NON	
Excentricité	excentricite	NUMBER(6,4)	OUI	NON	
Zone couverture	zone_couverture	VARCHAR2(200)	NON	NON	

Code	Règle de gestion (description complète)	Catégorie	Implémentation Oracle
RG-O01	Une orbite est une entité indépendante définie par ses paramètres physiques. Plusieurs satellites peuvent être positionnés sur une même orbite.	Structure relationnelle	Table ORBITE + FK
RG-O02	Deux orbites ne peuvent pas avoir simultanément les mêmes valeurs d'altitude et d'inclinaison, car elles représenteraient le même plan orbital.	Structure relationnelle	UNIQUE (altitude, inclinaison)
RG-O03	Une orbite peut exister dans la base sans être encore associée à un satellite (orbite planifiée).	Structure relationnelle	FK nullable côté SATELLITE

1.1.3 – Instrument

Attribut	Code	Type Oracle	Obligatoire	Unique	Contraintes
Référence	ref_instrument	VARCHAR2(20)	OUI	OUI	
Type	type_instrument	VARCHAR2(50)	OUI	NON	
Modèle	modele	VARCHAR2(100)	OUI	NON	
Résolution	resolution	NUMBER(6,1)	NON	NON	NULL possible
Consommation	consommation	NUMBER(5,2)	OUI	NON	
Masse	masse	NUMBER(5,3)	OUI	NON	

Code	Règle de gestion (description complète)	Catégorie	Implémentation Oracle
RG-I01	Un instrument est référencé dans un catalogue global indépendamment de son affectation à un satellite. Il est identifié par une référence unique.	Structure relationnelle	PK INSTRUMENT
RG-I02	Un même instrument peut être embarqué sur plusieurs satellites différents. Inversement, un satellite peut embarquer plusieurs instruments.	Structure relationnelle	Association N-N
RG-I03	Un instrument ne peut pas être physiquement embarqué simultanément sur deux satellites (contrainte d'unicité physique).	Mécanisme procédural	Trigger
RG-I04	Si un instrument est en état « Hors service » pendant plus de 30 jours, le satellite doit être signalé pour inspection.	Mécanisme procédural	Procédure PL/SQL

1.1.4 – Embarquement

Attribut	Code	Type	Obligatoire	Unique	Contraintes
Satellite	id_satellite	VARCHAR2(20)	OUI	PK	FK
Instrument	ref_instrument	VARCHAR2(20)	OUI	PK	FK
Date intégration	date_integration	DATE	OUI	NON	
État	etat_fonctionnement	VARCHAR2(20)	OUI	NON	Nominal / Dégradé / Hors service

1.1.5 – Station_Sol

Attribut	Code	Type Oracle	Obligatoire	Unique	Contraintes
Code station	code_station	VARCHAR2(20)	OUI	OUI	
Nom	nom_station	VARCHAR2(100)	OUI	NON	
Latitude	latitude	NUMBER(9,6)	OUI	NON	
Longitude	longitude	NUMBER(9,6)	OUI	NON	
Diamètre	diametre_antenne	NUMBER(4,1)	OUI	NON	
Bande	bande_frequence	VARCHAR2(10)	OUI	NON	
Débit max	debit_max	NUMBER(6,1)	OUI	NON	
Statut	statut	VARCHAR2(20)	OUI	NON	Active / Maintenance

Code	Règle de gestion (description complète)	Catégorie	Implémentation Oracle
RG-G01	Chaque station au sol est identifiée par un code unique et possède des coordonnées géographiques précises (latitude et longitude obligatoires).	Contrainte simple	PK + NOT NULL
RG-G02	Une station peut communiquer avec plusieurs satellites à des moments différents, et un satellite peut être en contact avec plusieurs stations.	Structure relationnelle	Association via FENETRE_COM
RG-G03	Une station en statut « Maintenance » ne peut pas planifier de nouvelles fenêtres de communication.	Mécanisme procédural	Trigger
RG-G04	Chaque station est rattachée à un unique centre de contrôle, mais un centre peut superviser plusieurs stations.	Structure relationnelle	FK + table d'affectation

1.1.6 – Centre_Controle

Attribut	Code	Type	Obligatoire	Unique	Contraintes
Identifiant	id_centre	VARCHAR2(20)	OUI	OUI	
Nom	nom_centre	VARCHAR2(100)	OUI	NON	
Ville	ville	VARCHAR2(50)	OUI	NON	
Région	region_geo	VARCHAR2(50)	OUI	NON	
Fuseau	fuseau_horaire	VARCHAR2(50)	OUI	NON	
Statut	statut	VARCHAR2(20)	OUI	NON	

1.1.7 – Affectation Station

Attribut	Code	Type	Obligatoire	Unique	Contraintes
Centre	id_centre	VARCHAR2(20)	OUI	PK	FK
Station	code_station	VARCHAR2(20)	OUI	PK	FK
Date	date_affectation	DATE	OUI	NON	

1.1.8 – Mission

Attribut	Code	Type	Obligatoire	Unique	Contraintes
Identifiant	id_mission	VARCHAR2(20)	OUI	OUI	
Nom	nom_mission	VARCHAR2(100)	OUI	NON	
Objectif	objectif	VARCHAR2(500)	OUI	NON	
Zone	zone_geo_cible	VARCHAR2(200)	OUI	NON	
Date début	date_debut	DATE	OUI	NON	
Date fin	date_fin	DATE	NON	NON	NULL si active (RG-M01)
Statut	statut_mission	VARCHAR2(20)	OUI	NON	

Code	Règle de gestion (description complète)	Catégorie	Implémentation Oracle
RG-M01	Une mission est identifiée par un code unique. Elle possède obligatoirement une date de début. La date de fin est facultative (NULL si mission active).	Contrainte simple	PK + NOT NULL / NULL
RG-M02	Une mission mobilise au moins un satellite. Un satellite peut participer à plusieurs missions.	Structure relationnelle	Association N-N
RG-M03	Pour chaque participation à une mission, le rôle du satellite doit être enregistré (ex. imageur, relais, secours).	Structure relationnelle	Attribut dans PARTICIPATION
RG-M04	Une mission ayant le statut « Terminée » ne peut plus accepter de nouveaux satellites.	Mécanisme procédural	Trigger

1.1.9 – Fenêtre_com

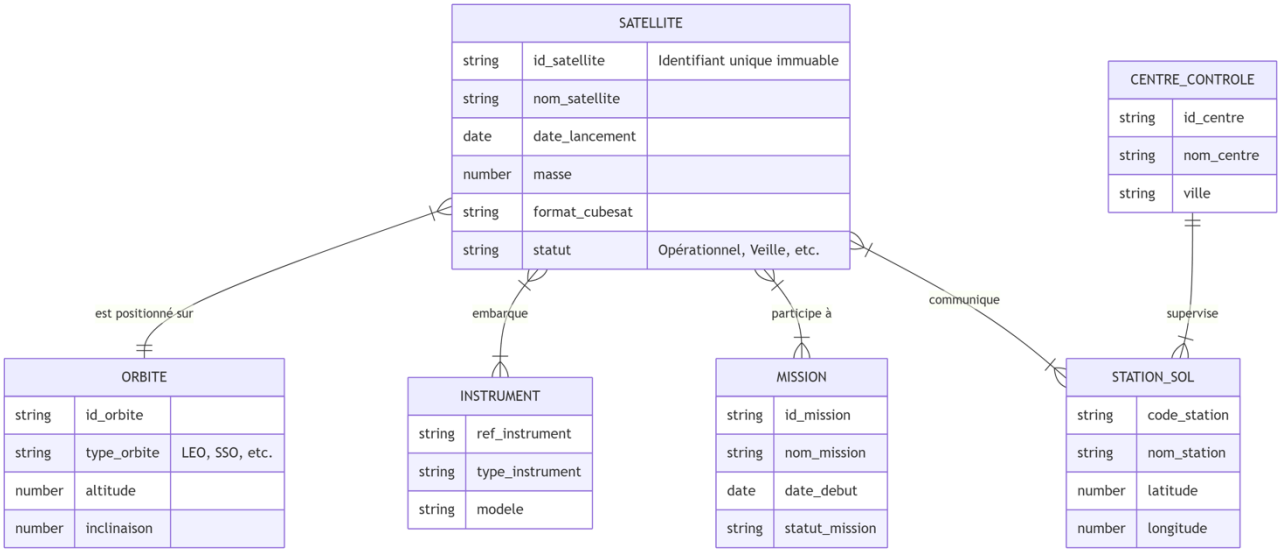
Attribut	Code	Type	Obligatoire	Unique	Contraintes
Identifiant	id_fenetre	NUMBER	OUI	OUI	Auto-incrément
Début	datetime_debut	TIMESTAMP	OUI	NON	
Durée	duree	NUMBER(4)	OUI	NON	1–900 (RG-F04)
Élévation	elevation_max	NUMBER(5,2)	OUI	NON	
Volume	volume_donnees	NUMBER(8,1)	NON	NON	NULL si planifiée
Statut	statut	VARCHAR2(20)	OUI	NON	
Satellite	id_satellite	VARCHAR2(20)	OUI	NON	FK
Station	code_station	VARCHAR2(20)	OUI	NON	FK

Code	Règle de gestion (description complète)	Catégorie	Implémentation Oracle
RG-F01	Une fenêtre de communication doit obligatoirement être associée à un satellite et à une station au sol.	Structure relationnelle	FK NOT NULL
RG-F02	Pour un même satellite, deux fenêtres de communication ne peuvent pas se chevaucher dans le temps.	Mécanisme procédural	Trigger
RG-F03	Pour une même station, deux fenêtres de communication ne peuvent pas se chevaucher dans le temps.	Mécanisme procédural	Trigger
RG-F04	La durée d'une fenêtre de communication doit être comprise entre 1 et 900 secondes.	Contrainte simple	CHECK
RG-F05	Le volume de données téléchargé ne peut être renseigné que si la fenêtre est au statut « Réalisée ». Il doit être NULL si la fenêtre est « Planifiée ».	Mécanisme procédural	Trigger

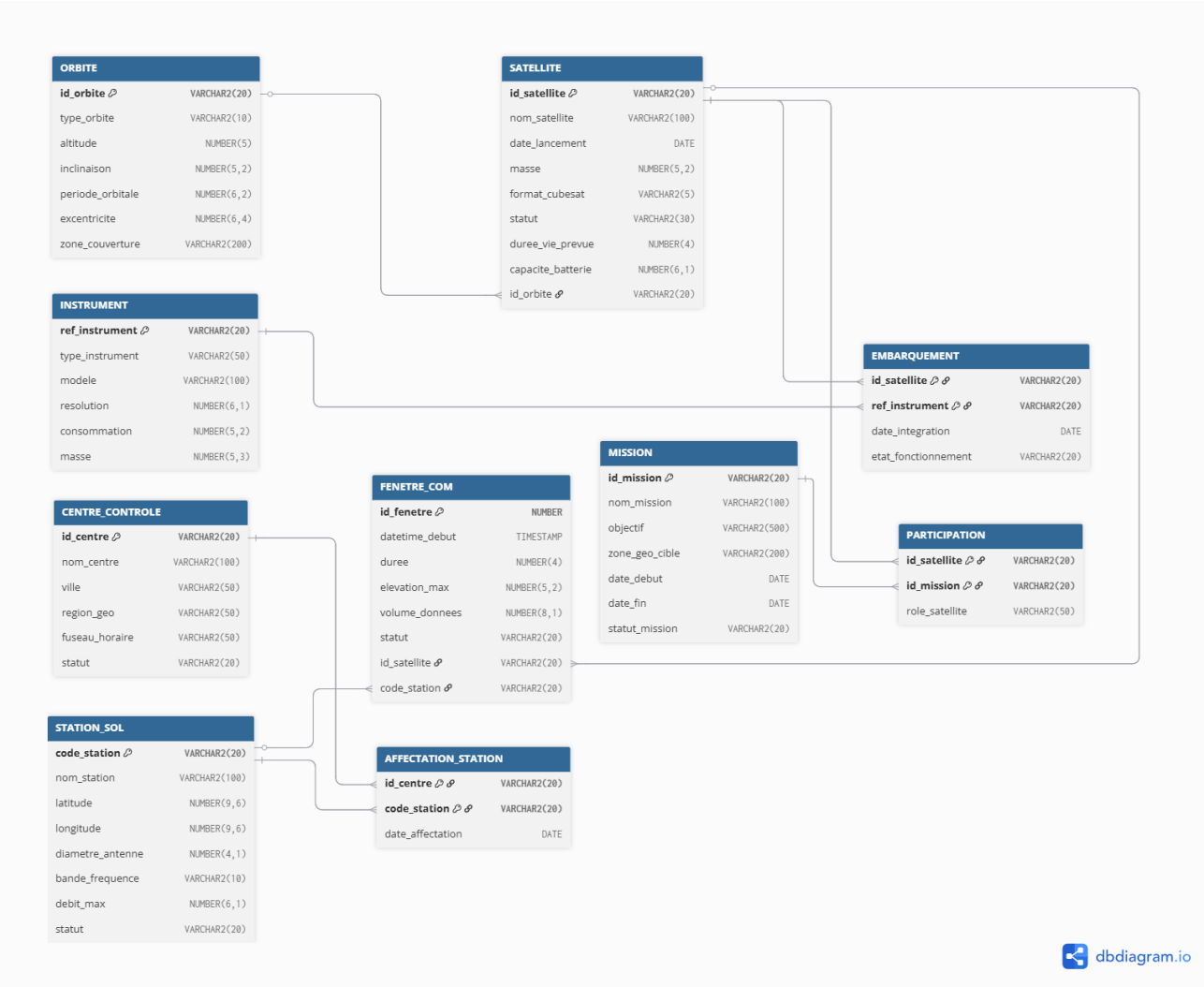
1.1.10 – Participation

Attribut	Code	Type	Obligatoire	Unique	Contraintes
Satellite	id_satellite	VARCHAR2(20)	OUI	PK	FK
Mission	id_mission	VARCHAR2(20)	OUI	PK	FK
Rôle	role_satellite	VARCHAR2(50)	OUI	NON	

1.2 – Étape 2 : Modèle Conceptuel de Données (MCD)



1.3 – Étape 3 : Modèle Logique de Données (MLD)



1.4 – Étape 4 : Réflexion sur l'architecture distribué

1.4.1 – Quelles tables du MLD sont strictement locales à un centre de contrôle ?

Les tables suivantes peuvent être considérées comme locales à un centre :

- **FENETRE_COM**
- **AFFECTATION_STATION**

Ces données sont fortement liées à l'activité opérationnelle locale :

FENETRE_COM

- correspond à la planification des communications entre satellites et stations
- dépend directement des stations gérées par le centre
- nécessite des décisions en temps réel

AFFECTATION_STATION

- reflète l'organisation interne (quel centre gère quelle station)
- spécifique à la structure opérationnelle

Ces données :

- N'ont pas besoin d'être modifiées globalement
- Sont propres à un centre donné
- Peuvent être consultées ailleurs mais pas forcément modifiées

1.4.2 – Quelles tables doivent être globales (accessibles depuis tous les centres) ?

Les tables suivantes doivent être **accessibles depuis tous les centres** :

- **SATELLITE**
- **ORBITE**
- **INSTRUMENT**
- **MISSION**
- **PARTICIPATION**
- **STATION_SOL** (au moins en lecture)

Ces données représentent le **référentiel métier global** :

- un satellite est utilisé par plusieurs centres
- une mission implique plusieurs centres
- les orbites et instruments sont communs

1.4.2.1 – Réplication complète (multi-maître)

Pour :

- SATELLITE
- MISSION
- PARTICIPATION

Permets-la lecture/écriture depuis chaque centre et une haute disponibilité

1.4.2.2 – Réplication en lecture seule

Pour :

- ORBITE
- INSTRUMENT

Données peu modifiées → cohérence facile

1.4.2.3 – Synchronisation périodique / quasi-temps réel

Pour :

- STATION_SOL

car statut (ex : maintenance) peut évoluer

1.4.3 – Comment le centre de Singapour peut-il continuer à planifier des fenêtres de communication si le serveur central est indisponible ?

Si le serveur central est indisponible, Singapour doit pouvoir continuer à :

- planifier des fenêtres
- accéder aux satellites et stations

Fragmentation horizontale :

On répartit **FENETRE_COM** par centre :

- Paris → fenêtres sur GS-TLS-01, GS-KIR-01
- Houston → ses stations
- Singapour → GS-SGP-01

Singapour possède **sa propre base locale complète pour ses données**

Réplication locale des données globales :

Singapour garde une copie locale de :
SATELLITE
MISSION
STATION_SOL
Donc même en cas de coupure la lecture et planification est possible

En cas de fonctionnement en mode dégradé les opérations locales sont autorisées et on procède à une synchronisation différée une fois le réseau rétabli

1.4.4 – Quels risques de cohérence identifiez-vous dans ce système multisite ?

Premier risque : mise à jour concurrente d'un satellite
Paris modifie le statut → "En veille"
Houston modifie en même temps → "Opérationnel"
On à donc un conflit de version et une incohérence globale

Deuxième risque : planification de fenêtres concurrentes
Singapour planifie une fenêtre pour SAT-001
Paris planifie une autre au même moment
On à donc une violation des règles RG-F02 et RG-F03 (chevauchement) et une incohérence métier

Autres risques possibles
Ajout d'un satellite à une mission terminée depuis un site non synchronisé
Utilisation d'une station en maintenance non mise à jour

2 – Phase 2 : Schéma Oracle & Triggers

2.1 – Génération du schéma DDL

2.1.1 – Q1 — Pourquoi ne peut-on pas créer SATELLITE avant ORBITE ? Quelle règle de gestion cela traduit-il ?

On ne peut pas créer **SATELLITE** avant **ORBITE** car la table **SATELLITE** contient une clé étrangère :
FOREIGN KEY (id_orbite) REFERENCES ORBITE(id_orbite)
Cela implique que toute valeur de id_orbite insérée dans SATELLITE doit exister préalablement dans ORBITE.
Donc, ORBITE est une table parente et SATELLITE une table **enfant**.
DONC
Un satellite doit obligatoirement être positionné sur une orbite existante (relation de dépendance / intégrité référentielle).

2.1.2 – Q2 — La règle RG-S06 (satellite désorbité : plus de fenêtre ni de mission) peut-elle être vérifiée au niveau DDL seul ? Quelle solution proposez-vous ?

Non, cette règle ne peut pas être entièrement vérifiée au niveau DDL.
Le DDL (contraintes CHECK, FK, NOT NULL) ne permet pas de vérifier des conditions inter-tables dynamiques ou de contrôler l'existence de lignes dans d'autres tables selon une logique métier globale.

Ici, il faut vérifier que : Si un satellite est désorbité (statut = 'désorbité' par exemple),
alors : aucune ligne dans FENETRE_COM ne lui est associée et aucune participation à une mission n'existe
Solution :
Utiliser des **triggers PL/SQL** (BEFORE INSERT/UPDATE/DELETE) pour :

- bloquer les insertions dans FENETRE_COM si le satellite est désorbité
- empêcher l'ajout de PARTICIPATION
- éventuellement ajouter des contrôles applicatifs (logique métier côté application)

2.1.3 – Comment implémentez-vous la contrainte RG-F02 (pas de chevauchement de fenêtres pour un même satellite) ? Cette contrainte est-elle directement exprimable en CHECK ?

Non, cette contrainte n'est pas exprimable via un CHECK.
Un CHECK ne peut pas comparer plusieurs lignes et ne peut pas interroger la table elle-même pour détecter des chevauchements
Or ici, il faut vérifier que pour un même id_satellite :
les intervalles temporels des fenêtres (datetime_debut + duree) ne se chevauchent pas. C'est une contrainte **inter-lignes + temporelle**.

Solution :
Utiliser un **trigger BEFORE INSERT/UPDATE** sur FENETRE_COM
dans le trigger :
Calculer la fin de fenêtre (datetime_debut + intervalle)
Vérifier qu'aucune fenêtre existante du même satellite n'intersecte avec la nouvelle

2.1.4 – Q4 — Quel type Oracle choisissez-vous pour format_cubesat (valeurs possibles : 1U, 3U, 6U, 12U) ? Justifiez.

Le type choisi est : VARCHAR2(5)
Justification :

- Les valeurs sont alphanumériques (1U, 3U, etc.)
- Elles ne sont pas purement numériques
- Le suffixe 'U' est significatif (unité de cube satellite)

VARCHAR2 est donc adapté pour stocker, des codes catégoriels, des valeurs non arithmétiques
Amélioration possible :
Ajouter une contrainte CHECK pour limiter les valeurs autorisées :
CONSTRAINT chk_format_cubesat
CHECK (format_cubesat IN ('1U', '3U', '6U', '12U'))

Cela garantit la cohérence des données tout en restant simple.

2.2 – Génération du DML

Voir sur le github, création des données conformément aux fichiers csv fournit dans les fichiers.

2.3 – Triggers

Voir sur le github, création des triggers conformément aux attendus + cas de test.

2.4 – Script control

Ajout d'un court script de contrôle permettant de vérifier rapidement les informations générales de la base de données.